

Q1

Why does the perceptron algorithm use class labels **-1 and +1** instead of **0 and 1**? Explain how this choice affects the perceptron update rule.

Answer : To reduce the extra logical calculation -1 and +1 are used instead of 0 and 1. The perceptron predicts based on the sign of the score :

$$\hat{y} = \text{sign}(w \cdot x)$$

With labels -1 and +1 correctness can be checked with a single condition:

$$y = \text{sign}(w \cdot x) > 0 \rightarrow \text{correct}$$

$$y = \text{sign}(w \cdot x) \leq 0 \rightarrow \text{mistake}$$

If we used 0,1 we wouldn't get this as simple, easy and clean. We would need extra logic, thresholding, mapping $0 \Rightarrow -1$ etc, which complicates both notation and updates.

From the update rule:

$$w = w + \eta y x$$

With -1,+1 , if $y=+1$ directs toward x , if $y=-1$ directs away from x , So a single formula handles both classes naturally.

Q2

Gradient descent usually updates weights by **subtracting** the gradient. Why does the perceptron update rule **add** the term (eta , y , x) instead?

Answer : Actually both do the same work .

In gradient descent we subtract the gradient, in perceptron the gradient turns out to be negative ,so subtracting a negative means adding.

$$w = w - (\text{negative value}) = w + \eta y x$$

When the model is wrong it adjusts weights in the direction that fixes the mistake.

Q3(optional)

Explain in your own words how the perceptron learning rule can be seen as a form of **gradient descent**. Also mention one limitation of the perceptron loss function.

Answer : Perceptron as gradient descent :

- I. It tries to reduce mistake step by step
- II. It updates weights based on error

So we can assume it a simple version of gradient descent

Limitations of perceptron's loss function :

- I. It only cares if a point is right or wrong
- II. It does not care how confidently it is right, so even if a point is barely correct, it stops learning from it
This makes the model less accurate compared to better methods.